

NAME : J.ANJALI

BATCHNO: 05

ROLLNO: 2403A510FO

COURSE: AI ASSISSTED CODING

TASK1:

Lab Question 1: Sorting Algorithm Translation

You are part of a multinational development team. The backend is written in Java, but a new module requires a Python implementation of the same algorithm for integration with a data science pipeline.

- Task 1: Use AI-assisted coding to translate a given Java bubble sort program into Python. Verify that the translated code works correctly.
- Task 2: Introduce errors in the Python version to check if the input list is empty or contains non-numeric values.

PROMPT:

"Translate the following Java bubble sort implementation into a clean, idiomatic Python function. The Python function should accept a list of numbers and return the sorted list. Please include a simple example to verify that the translated function works correctly."

CODE:

```

19.1.py > bubble_sort_robust
1 def bubble_sort_robust(data: list) -> list:
2     """
3     Sorts a list of numbers using the bubble sort algorithm with error handling.
4
5     Args:
6     | data: A list of numbers to be sorted.
7
8     Returns:
9     | A new list containing the sorted numbers.
10
11     Raises:
12     | ValueError: If the input list is empty or contains non-numeric values.
13     """
14     # Task 2: Check if the input list is empty.
15     if not data:
16         raise ValueError("Input list cannot be empty.")
17
18     # Create a copy to avoid modifying the original list.
19     arr = list(data)
20     n = len(arr)
21
22     # Task 2: Check for non-numeric values before sorting.
23     for item in arr:
24         if not isinstance(item, (int, float)):
25             raise ValueError(f"Input list contains a non-numeric value: '{item}'.")
26
27     # Task 1: Translated bubble sort logic with a small optimization.
28     for i in range(n - 1):
29         swapped = False
30         for j in range(0, n - i - 1):

```

```

31             if arr[j] > arr[j + 1]:
32                 # Python's idiomatic way to swap elements.
33                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
34                 swapped = True
35             # If no two elements were swapped by inner loop, the list is sorted.
36             if not swapped:
37                 break
38
39     return arr
40
41 # Example usage
42 if __name__ == "__main__":
43     print("---- Bubble Sort with Error Handling ----\n")
44
45     # 1. Verification of the sorting algorithm
46     print("---- Test Case 1: Standard Sort ----")
47     unsorted_list = [64, 34, 25, 12, 22, 11, 90]
48     print(f"Original list: {unsorted_list}")
49     try:
50         sorted_list = bubble_sort_robust(unsorted_list)
51         print(f"Sorted list: {sorted_list}\n")
52     except ValueError as e:
53         print(f"Error: {e}\n")
54
55     # 2. Test for the empty list error
56     print("---- Test Case 2: Empty List ----")
57     empty_list = []
58     print(f"Original list: {empty_list}")
59     try:

```

```

60         bubble_sort_robust(empty_list)
61     except ValueError as e:
62         print(f"Caught expected error: {e}\n")
63
64     # 3. Test for the non-numeric value error
65     print("---- Test Case 3: Non-Numeric Data ----")
66     invalid_list = [10, 40, 'fifty', 20]
67     print(f"Original list: {invalid_list}")
68     try:
69         bubble_sort_robust(invalid_list)
70     except ValueError as e:
71         print(f"Caught expected error: {e}\n")
72

```

Output:

```

PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/19.1.py
--- Bubble Sort with Error Handling ---

--- Test Case 1: Standard Sort ---
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]

--- Test Case 1: Standard Sort ---
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
--- Test Case 1: Standard Sort ---
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]

--- Test Case 2: Empty List ---

--- Test Case 2: Empty List ---
Original list: []
Caught expected error: Input list cannot be empty.
--- Test Case 2: Empty List ---
Original list: []
Caught expected error: Input list cannot be empty.
Original list: []
Caught expected error: Input list cannot be empty.
Caught expected error: Input list cannot be empty.

--- Test Case 3: Non-Numeric Data ---
Original list: [10, 40, 'fifty', 20]
Caught expected error: Input list contains a non-numeric value: 'fifty'.

```

Observation:

The `bubble_sort_robust` function effectively sorts a list of numbers using the bubble sort algorithm. A key observation is its strong emphasis on input validation: it explicitly checks for and raises `ValueError` if the input list is empty or contains non-numeric elements, making it robust against common data issues. The implementation is clear, utilizing Pythonic swaps and including an optimization to stop early if the list becomes sorted. It also employs a defensive copy of the input list, preventing unintended side effects. While bubble sort is generally inefficient for large datasets ($O(n^2)$), the added error handling and clear structure make this a well-engineered solution for its specific requirements. [cite: 19.1]

TASK2:

Lab Question 2: File Handling Translation

A company's legacy codebases stores and processes files in C++, but the analytics team needs an equivalent program in JavaScript (Node.js) for integration with a web dashboard.

- Task 1: Translate a given C++ file read-and-write program into JavaScript using AI assistance. Ensure the script reads a text file and writes processed output to a new file.
- Task 2: Add error handling in the JavaScript version to gracefully handle missing files or permission errors

PROMPT:

"Translate the following C++ file I/O code to a Node.js script. The script should read `input.txt`, convert its contents to uppercase, and write to `output.txt`. Add error handling to the Node.js script for missing files and permission errors."

CODE:

```
JS 19.1.js > ...
1  const fs = require('fs/promises');
2  const path = require('path');
3
4  /**
5   * Reads content from an input file, converts it to uppercase, and writes it to an output file.
6   * This is a JavaScript (Node.js) translation of a common C++ file I/O task.
7   *
8   * @param {string} inputFilePath - The path to the source file.
9   * @param {string} outputFilePath - The path to the destination file.
10  * @returns {Promise<void>} A promise that resolves when the operation is complete.
11  */
12  async function processFile(inputFilePath, outputFilePath) {
13    try {
14      // --- Task 1: Read the input file ---
15      // In Node.js, we use the 'fs/promises' module for modern async file operations.
16      console.log(Reading from ${inputFilePath}...);
17      const fileContent = await fs.readFile(inputFilePath, 'utf-8');
18
19      // --- Process the data (equivalent to the C++ std::transform) ---
20      const processedContent = fileContent.toUpperCase();
21
22      // --- Task 1: Write the processed data to the output file ---
23      console.log(Writing processed data to ${outputFilePath}...);
24      await fs.writeFile(outputFilePath, processedContent);
25
26      console.log('✅ File processing completed successfully.');
```

```

35     console.error(` Error: The file at '${error.path}' was not found.`);
36 } else if (error.code === 'EACCES') {
37     // 'EACCES' means 'Error, Access Denied' (i.e., permission error).
38     console.error(` Error: Permission denied. Could not read or write to '${error.path}'.`);
39 } else {
40     // For any other unexpected errors.
41     console.error(` An unexpected error occurred:`, error.message);
42 }
43 }
44 }
45
46 // --- Main Execution and Demonstration ---
47 // This self-invoking async function allows us to use 'await' at the top level.
48 (async () => {
49     const inputFile = path.join(__dirname, 'input.txt');
50     const outputFile = path.join(__dirname, 'output.txt');
51
52     // --- 1. Demonstrate the successful case ---
53     console.log('--- Running Success Scenario ---');
54     // First, create a dummy input file to ensure the script can run.
55     await fs.writeFile(inputFile, 'Hello World!\nThis is a test file for the Node.js script.');
```

```

56     await processFile(inputFile, outputFile);
57     console.log('-----\n');
58
59     // --- 2. Demonstrate the "File Not Found" error ---
60     console.log('--- Running "File Not Found" Scenario ---');
61     const nonExistentFile = path.join(__dirname, 'non_existent_file.txt');
62     await processFile(nonExistentFile, outputFile);
63     console.log('-----\n');
64
65
66
67     // --- 3. Demonstrate the "Permission Denied" error (conceptual) ---
68     // This is harder to reliably test, but the logic is in place.
69     // To test this manually, you could:
70     // - On Linux/macOS: chmod 000 input.txt to remove read permissions.
71     // - On Windows: Change the file's security properties to deny read access.
72     console.log('--- "Permission Denied" Scenario (Conceptual) ---');
73     console.log('The code includes a check for EACCES (permission) errors.');
```

```

74     console.log('To test, manually restrict read permissions on "input.txt" and re-run.');
```

```

75     console.log('-----\n');
76
77     // --- Cleanup ---
78     // Clean up the created files.
79     try {
80         await fs.unlink(inputFile);
81         await fs.unlink(outputFile);
82     } catch (err) {
83         // Ignore cleanup errors if files were not created due to an earlier failure.
84     }
85 })();

```

OUTPUT:


```
[Running] node "c:\Users\DELL\Desktop\vs code\file_processor.js"
--- Running Success Scenario ---
Reading from c:\Users\DELL\Desktop\vs code\input.txt...
Writing processed data to c:\Users\DELL\Desktop\vs code\output.txt...
✅ File processing completed successfully.
-----

--- Running "File Not Found" Scenario ---
Reading from c:\Users\DELL\Desktop\vs code\non_existent_file.txt...
❌ An error occurred during file processing:
-----

Error: The file at 'c:\Users\DELL\Desktop\vs code\non_existent_file.txt' was not found.
--- "Permission Denied" Scenario (Conceptual) ---
The code includes a check for EACCES (permission) errors.
To test, manually restrict read permissions on "input.txt" and re-run.
-----
```

OBSERVATION:

The JavaScript script `19.1.js` is an exemplary Node.js solution for file processing, effectively translating a C++ file I/O task into modern, idiomatic JavaScript. It skillfully uses `async/await` with the `fs/promises` module, resulting in clean, non-blocking, and highly readable code.

A key strength is its robust error handling. The `try...catch` block doesn't just catch generic exceptions; it inspects the error code to provide specific, user-friendly feedback for common issues like a missing file (`ENOENT`) or permission denial (`EACCES`).

Furthermore, the script is self-contained and demonstrates best practices through its main execution block. It programmatically sets up and runs both a success scenario and a "file not found" failure case, and includes a cleanup step to remove generated files. This thoughtful structure makes the script not only functional but also easy to understand, test, and maintain, showcasing a professional approach to software development.

TASK3 :

Lab Question 3: API Call Translation

Your team developed a prototype in Python to fetch weather data from an API, but the production environment only supports Java.

- Task 1: Translate the Python script (that makes an API call and prints the response) into Java using AI-assisted coding. Ensure equivalent functionality.
- Task 2: Add proper error handling in the Java version for cases such as invalid API key, request timeout, or no internet connection.

PROMPT:

"Translate the following Python script into a Java program. The Python script fetches weather data from the OpenWeatherMap API, takes an API key and city as input, and prints the current temperature and description.

CODE:

```
1  import java.io.IOException;
2  import java.net.URI;
3  import java.net.http.HttpClient;
4  import java.net.http.HttpRequest;
5  import java.net.http.HttpResponse;
6  import java.net.http.HttpTimeoutException;
7  import java.time.Duration;
8  import org.json.JSONObject;
9
10 /**
11  * A Java client to fetch weather data from the OpenWeatherMap API.
12  * This is a translation of a common Python task, with robust error handling.
13  */
14 public class WeatherApiClient {
15
16     // --- Configuration ---
17     // It's best practice to use an environment variable for the API key.
18     // Replace "YOUR_API_KEY" with your actual key if not using environment variables.
19     private static final String API_KEY = System.getenv().getOrDefault("OPENWEATHER_API_KEY", "YOUR_API_KEY");
20     private static final String API_BASE_URL = "https://api.openweathermap.org/data/2.5/weather";
21     private static final Duration REQUEST_TIMEOUT = Duration.ofSeconds(5);
22
23     /**
24      * Fetches weather data for a specified city.
25      *
26      * @param city The name of the city (e.g., "London").
27      */
28     public static void fetchWeather(String city) {
29         // --- Task 1: Build the API Request ---
30         // Create a modern HttpClient instance.
31         HttpClient client = HttpClient.newBuilder()
32             .connectTimeout(REQUEST_TIMEOUT)
```

```

33         .build();
34
35         // Construct the full URL with query parameters.
36         String fullUrl = String.format("%s?q=%s&appid=%s&units=metric", API_BASE_URL, city, API_KEY);
37
38         // Create an HTTP GET request object.
39         HttpRequest request = HttpRequest.newBuilder()
40             .uri(URI.create(fullUrl))
41             .timeout(REQUEST_TIMEOUT) // Set a timeout for the entire request-response cycle.
42             .GET()
43             .build();
44
45         System.out.println("Fetching weather data for " + city + "...");
46
47         try {
48             // --- Task 1: Send the request and get the response ---
49             // The response body is handled as a String.
50             HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
51
52             // --- Task 2: Add Error Handling ---
53             int statusCode = response.statusCode();
54             if (statusCode == 200) {
55                 // Success: Parse and print the JSON response.
56                 JSONObject weatherData = new JSONObject(response.body());
57                 System.out.println("✅ API Call Successful. Response:");
58                 // Use an indent factor for pretty-printing the JSON.
59                 System.out.println(weatherData.toString(4));
60             } else if (statusCode == 401) {
61
62             } else if (statusCode == 401) {
63                 // Specific error for an invalid API key.
64                 System.err.println("❌ Error: Invalid API Key. Please check your key and permissions.");
65             } else {
66                 // Handle other unsuccessful HTTP status codes (e.g., 404 Not Found, 500 Server Error).
67                 System.err.println("❌ Error: Received HTTP status code " + statusCode);
68             }
69         } catch (HttpTimeoutException e) {
70             // Specific error for request timeouts.
71             System.err.println("❌ Error: The request timed out after " + REQUEST_TIMEOUT.toSeconds() + " seconds.");
72         } catch (IOException | InterruptedException e) {
73             // General error for network issues (e.g., no internet connection) or thread interruption.
74             System.err.println("❌ Error: A network or connection error occurred. Check your internet connection.");
75             // Optional: print stack trace for debugging.
76             // e.printStackTrace();
77         }
78     }
79
80     /**
81      * Main method to execute the weather fetch operation.
82      */
83     public static void main(String[] args) {
84         if (API_KEY == null || API_KEY.equals("YOUR_API_KEY")) {
85             System.err.println("Error: API key is not configured.");
86             System.err.println("Please set the OPENWEATHER_API_KEY environment variable or replace the placeholder in the code.");
87             return;
88         }
89
90         // --- Demonstrate a successful API call ---
91         System.out.println("--- Running Success Scenario ---");
92         fetchWeather("London");
93         System.out.println("-----\n");
94
95         // --- Demonstrate an error scenario (e.g., invalid city) ---
96         // The API will return a 404 Not Found error.
97         System.out.println("--- Running 'Not Found' Scenario ---");
98         fetchWeather("InvalidCityName123");
99         System.out.println("-----\n");
100     }
101 }

```

OUTPUT:


```

WeatherData.java
1  --- Weather Reporter ---
2  Error: API key is missing or not configured. Please provide a valid OpenWeatherMap API key.
3
4  --- Testing with an invalid city ---
5  Error: API key is missing or not configured. Please provide a valid OpenWeatherMap API key.
6
7  --- Testing with an invalid API key ---
8  Fetching latest weather data for London...
9  Error: Invalid API key. Please check your key and try again.
10 |
    --- Weather Reporter ---

```

OBSERVATION:

- **Translation and Verbosity:** The translation from a Python script using the `requests` library to a Java program is a clear demonstration of how high-level libraries can simplify code. The Java version is significantly more verbose. It requires explicit setup of an `HttpClient` and `HttpRequest`, manual checking of status codes, and the use of a separate library (`org.json`) for parsing, all of which are handled more concisely in Python.
- **Error Handling (Task 2):** Java's static typing and checked exceptions force a more structured approach to error handling. The `try-catch` block is more complex, needing to catch specific exceptions like `HttpTimeoutException`, `IOException`, and `InterruptedException`. This makes the error handling logic very explicit and robust, as the compiler ensures these potential issues are addressed. In contrast, Python's dynamic nature makes it easier to write the initial code but requires more discipline to handle all possible runtime exceptions.
- **Dependencies:** A key difference is dependency management. Python's `requests` is a standard de-facto library, but JSON parsing is built-in. The Java solution requires an external dependency (`org.json`) for a core part of the task (JSON parsing), which adds a step to the project setup (e.g., configuring Maven or Gradle).